Gradient Descent and Automatic Differentiation

Seminar — Week 3: key theory and a hands-on exercise

Optimization Methods

DSBA

Plan

Part	Minutes
I. Gradient descent: step sizes, line search, stopping, rates	20
II. Automatic differentiation: forward and reverse mode	15
III. Exercise: implement gradient descent by hand	40
Wrap-up and discussion	5

Results are stated **without proofs**; the proofs are in the lecture notes. Focus on *what* each result says and *how* to use it in code.

Part I

Gradient descent

Iterative methods: the common template

Starting from x_0, repeat

$$x_{k+1} = x_k + \alpha_k d_k.$$

Every method answers three design questions:

- **direction** d_k — where to move;
- **step size** $\alpha_k > 0$ — how far to move;
- **termination** — when to stop.

Descent direction

d is a (strict) descent direction at x if $\nabla f(x)^\top d < 0$. Then $f(x + \alpha d) < f(x)$ for all sufficiently small $\alpha > 0$.

Reason: by first-order Taylor expansion, $f(x + td) = f(x) + t\,\nabla f(x)^\top d + o(t)$, so for small t the sign of $\nabla f(x)^\top d$ decides whether f goes up or down.

Why the negative gradient?

Steepest descent (Euclidean norm)

Among all directions with $\|d\|_2 = 1$, the directional derivative $\nabla f(x)^\top d$ is smallest for

$$d = -\frac{\nabla f(x)}{\|\nabla f(x)\|_2}, \qquad \text{with value } -\|\nabla f(x)\|_2 .$$

Intuition. $\nabla f(x)^\top d = \|\nabla f(x)\| \, \|d\| \cos\theta$, and only the angle θ depends on d; the minimum is at $\theta = 180°$.

The answer depends on how we measure length. If length is measured by $\|d\|_M^2 = d^\top M d$ with $M \succ 0$, the steepest direction becomes

$$d \propto -M^{-1}\nabla f(x).$$

This is the idea behind *preconditioning*, Newton's method ($M = \nabla^2 f$) and natural gradient.

Gradient descent and smoothness

Gradient descent with a constant step

$$x_{k+1} = x_k - \alpha \nabla f(x_k), \qquad \alpha > 0.$$

One iteration costs one gradient evaluation.

L-smoothness: $\|\nabla f(x) - \nabla f(y)\| \le L \|x - y\|$. For twice-differentiable f: all Hessian eigenvalues are at most L (curvature $\le L$).

Descent lemma. For L-smooth f,

$$f(y) \le f(x) + \nabla f(x)^\top (y - x) + \tfrac{L}{2} \|y - x\|^2 .$$

Consequence for one GD step:

$$f(x_{k+1}) \le f(x_k) - \alpha \Big(1 - \tfrac{L\alpha}{2}\Big) \|\nabla f(x_k)\|^2 .$$

- $0 < \alpha < 2/L$: guaranteed decrease whenever $\nabla f(x_k) \neq 0$;
- $\alpha = 1/L$: the best guarantee, $f(x_{k+1}) \le f(x_k) - \frac{1}{2L} \|\nabla f(x_k)\|^2$.

Armijo backtracking line search

When L is unknown, choose the step adaptively. Parameters: $\alpha_0 > 0$, $\beta \in (0,1)$, $c \in (0,1)$; typical $\beta = 0.5$, $c = 10^{-4}$.

Algorithm

1. $\alpha \leftarrow \alpha_0$; compute $f(x)$ and $\nabla f(x)^\top d$ *once*;
2. while $f(x + \alpha d) > f(x) + c\,\alpha\,\nabla f(x)^\top d$: $\alpha \leftarrow \beta\alpha$;
3. return α.

For GD, $d = -\nabla f(x)$ and the test reads $f(x - \alpha\nabla f) \le f(x) - c\,\alpha\,\|\nabla f(x)\|^2$.

- **Meaning:** accept a step only if it achieves at least a fraction c of the decrease predicted by the linear model ("sufficient decrease").
- **Termination:** for L-smooth f the test holds for all $\alpha \le 2(1-c)/L$, so the loop always ends.
- **Caveat:** backtracking only *shrinks*; it never returns a step larger than α_0.
- **In code:** reject non-descent d, cap the number of shrinks, no gradient calls inside the loop.

Stopping criteria: use more than one signal

Scale-aware tests (relative for large values, absolute near zero):

$$\|\nabla f(x_k)\| \le \varepsilon_g \max(1, |f(x_k)|) \qquad \text{(gradient)}$$
$$\|x_{k+1} - x_k\| \le \varepsilon_x \max(1, \|x_k\|) \qquad \text{(step)}$$
$$|f(x_{k+1}) - f(x_k)| \le \varepsilon_f \max(1, |f(x_k)|) \qquad \text{(function change)}$$

Always add safeguards: maximum number of iterations / evaluations, and a check for NaN or Inf.

- A **small gradient** is the strongest first-order certificate of (near-)stationarity.
- Small changes in x or f may mean convergence — or just a tiny step, bad scaling or floating-point stagnation.
- Report *why* the method stopped: converged / max_iter / diverged / stagnated.

Part II

Automatic differentiation

Three ways to compute derivatives

Method	Idea	Accuracy	Typical issue
Symbolic	manipulate formulas	exact	expression swell
Finite differences	perturb inputs	approximate	truncation vs. round-off
Automatic (AD)	chain rule on the program	machine precision	memory, control flow

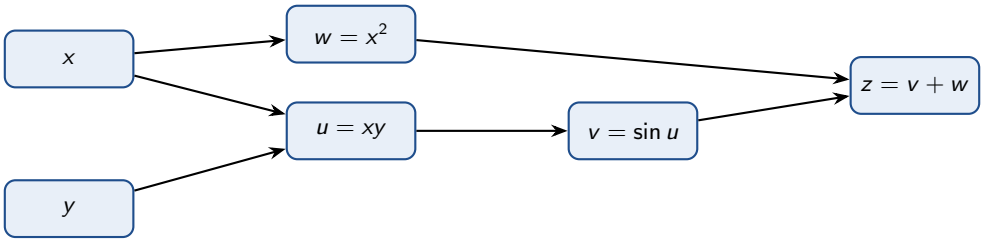
Central differences (used to *check* gradients):

$$\frac{\partial f}{\partial x_i} \approx \frac{f(x + he_i) - f(x - he_i)}{2h}.$$

Error $\approx$ truncation $O(h^2)$ + round-off $O(\varepsilon_{\text{mach}}/h)$; the best h is around 10^{-5} in double precision. One gradient costs $2n$ function evaluations, so it is a test, not a training tool.

A computation is a directed acyclic graph

Example: $z = \sin(xy) + x^2$. Intermediate variables: $u = xy$, $w = x^2$, $v = \sin u$, $z = v + w$.



- Nodes store values; edges encode dependencies; each node knows its *local* derivatives.
- The chain rule composes local derivatives along paths.
- x reaches z along **two** paths (through u and through w) — this matters later.

At $(x, y) = (2, -1)$: $u = -2$, $w = 4$, $v = \sin(-2) \approx -0.9093$, $z \approx 3.0907$.

Forward mode: propagate tangents

Choose an input direction $(\dot{x}, \dot{y})$. Each node carries its value and its tangent $\dot{v} = \frac{dv}{dt}$ along that direction:

$$\dot{u} = y\,\dot{x} + x\,\dot{y}, \qquad \dot{w} = 2x\,\dot{x}, \qquad \dot{v} = \cos(u)\,\dot{u}, \qquad \dot{z} = \dot{v} + \dot{w}.$$

At $(2, -1)$ with $(\dot{x}, \dot{y}) = (1, 0)$:

$$\dot{u} = -1, \quad \dot{w} = 4, \quad \dot{v} = \cos(-2) \cdot (-1) \approx 0.4161, \quad \dot{z} \approx 4.4161 = \frac{\partial z}{\partial x}.$$

- One forward sweep gives one Jacobian–vector product $J\dot{x}$.
- To get $\partial z/\partial y$ we need a *second* sweep with $(\dot{x}, \dot{y}) = (0, 1)$.
- Efficient when the number of inputs is small.

Reverse mode: propagate adjoints

Define the adjoint $\bar{v} = \partial z / \partial v$. Run the forward pass and **store** the values, then visit nodes in reverse topological order, starting from $\bar{z} = 1$:

$$\text{for every edge } a \to c: \qquad \bar{a} \mathrel{+}= \bar{c}\, \frac{\partial c}{\partial a}.$$

At $(2, -1)$, visiting z, v, w, u, y, x:

$$\bar{z} = 1, \quad \bar{v} = 1, \quad \bar{w} = 1, \quad \bar{u} = \bar{v} \cos u \approx -0.4161,$$

$$\bar{x} = \underbrace{\bar{u}\, y}_{\text{via } u} + \underbrace{\bar{w} \cdot 2x}_{\text{via } w} \approx 0.4161 + 4 = 4.4161, \qquad \bar{y} = \bar{u}\, x \approx -0.8323.$$

- One reverse sweep gives the *whole* gradient: a vector–Jacobian product $\bar{z}^\top J$.
- $\bar{x}$ is a **sum** of two path contributions: adjoints must **accumulate** (+=), never be assigned. Example: $z = x^2 + x^3 \Rightarrow dz/dx = 2x + 3x^2$.
- Frameworks also accumulate across calls: reset gradients before each backward pass (`zero_grad()` in PyTorch).

Forward vs. reverse, and backpropagation

For $F : \mathbb{R}^n \to \mathbb{R}^m$:

Mode	Sweeps for the full Jacobian	Best when
Forward	n	few inputs
Reverse	m	few outputs (e.g. a scalar loss)

A training loss is scalar ($m = 1$) with millions of parameters, so **backpropagation = reverse mode**. Its price is memory for stored intermediates (checkpointing trades recomputation for memory).

Example: logistic regression. $s_i = w^\top x_i + b$, $p_i = \sigma(s_i)$, $\ell_i = -y_i \log p_i - (1 - y_i) \log(1 - p_i)$. Reverse mode gives

$$\frac{\partial \ell_i}{\partial s_i} = p_i - y_i, \qquad \nabla_w \ell_i = (p_i - y_i)\, x_i, \qquad \frac{\partial \ell_i}{\partial b} = p_i - y_i.$$

Non-smooth primitives (ReLU, $|x|$ at 0) use a chosen subgradient convention.

Part III

Exercise: gradient descent in Python

Exercise: implement gradient descent

Test function.

$$f(x_1, x_2) = (x_1 - 3)^2 + 5\,(x_2 + 1)^2, \qquad x_0 = (0,\, 0).$$

Its minimizer is $x^* = (3,\, -1)$ with $f^* = 0$, so you can check your answer.

Your task.

1. Derive ∇f by hand and find the smoothness constant L (the largest eigenvalue of the Hessian).
2. Implement `gradient_descent(f, grad, x0, step, tol=1e-8, max_iter=10_000)`:
   - update $x_{k+1} = x_k - \alpha \nabla f(x_k)$;
   - stop when $\|\nabla f(x_k)\| \le$ `tol` or after `max_iter` iterations;
   - stop with an error if f or ∇f becomes NaN or Inf;
   - return the final point and a history of x_k, $f(x_k)$, $\|\nabla f(x_k)\|$ and the stop reason.
3. Run it with $\alpha = 1/L$ and check that it reaches x^*. Report the number of iterations and the final $f(x_k) - f^*$.
4. Repeat with $\alpha \in \{0.05,\ 0.19,\ 0.21\}$ and plot $f(x_k) - f^*$ vs. k on a log scale for all four steps. Explain what you see using the result "$0 < \alpha < 2/L$ guarantees decrease" from the smoothness slide.

Rules: NumPy and Matplotlib only; no optimizer libraries.

Starter code

```
import numpy as np
import matplotlib.pyplot as plt

def f(x):
    return (x[0] - 3)**2 + 5 * (x[1] + 1)**2

def grad(x):
    # TODO: gradient derived by hand
    ...

def gradient_descent(f, grad, x0, step, tol=1e-8, max_iter=10_000):
    x = np.asarray(x0, dtype=float).copy()
    history = {'x': [x.copy()], 'f': [f(x)], 'grad_norm': [], 'status': 'max_iter'}
    for k in range(max_iter):
        # TODO: gradient, stopping test, update, NaN/Inf checks, history
        ...
    return x, history

x0 = np.array([0.0, 0.0])
# TODO: run with step = 1/L, then 0.05, 0.19, 0.21, and plot f(x_k) - f*
```